Design the Reddit API

SystemsExpert design walkthrough

Clarifying questions

- Which entities: subreddits, posts, comments, votes, awards?
- Is this about API shape more than infrastructure?
- Pagination and sorting requirements?

Functional requirements

- Subreddits: create, get, list posts
- Posts: create, edit, delete, vote
- Comments: threaded create/edit/delete/vote
- Sorting: hot, new, top

Non-functional requirements

- Consistent, versioned, paginated REST resources
- Idempotent votes

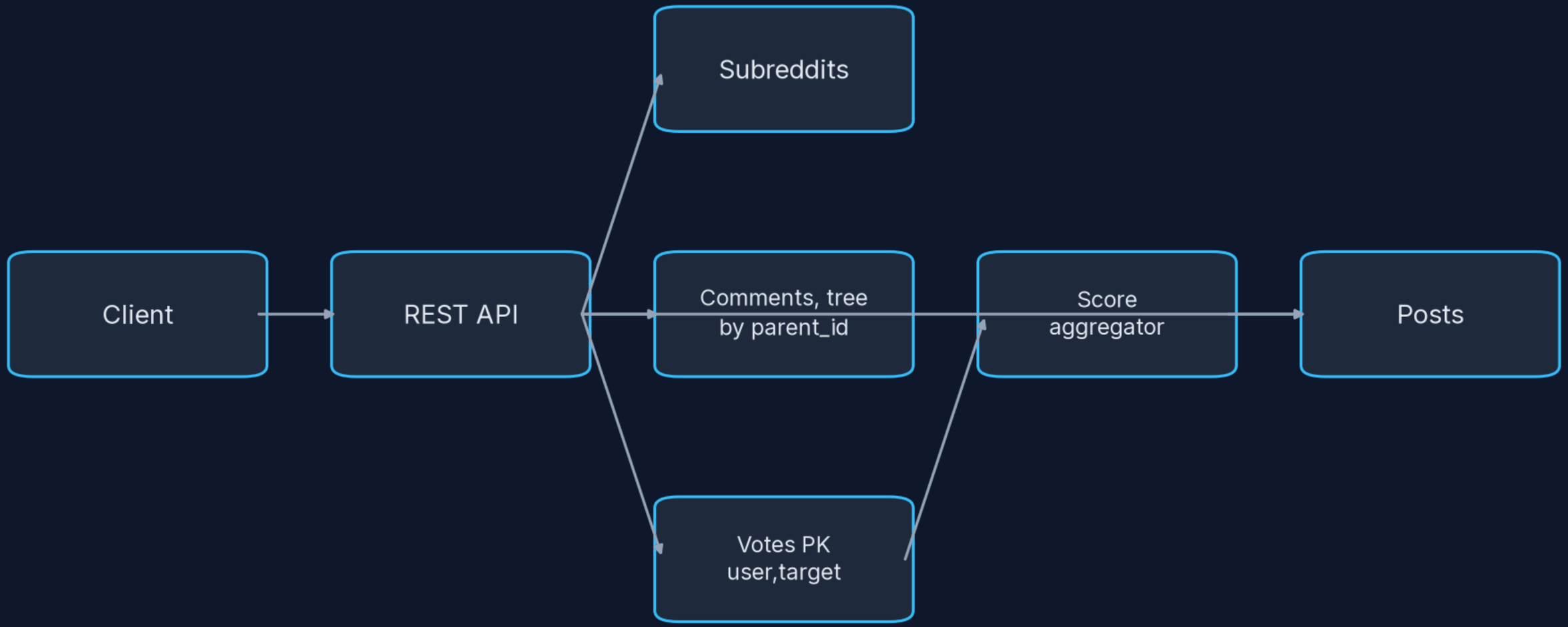
Capacity estimates

- Read-heavy (~100:1); vote writes are the hottest write path

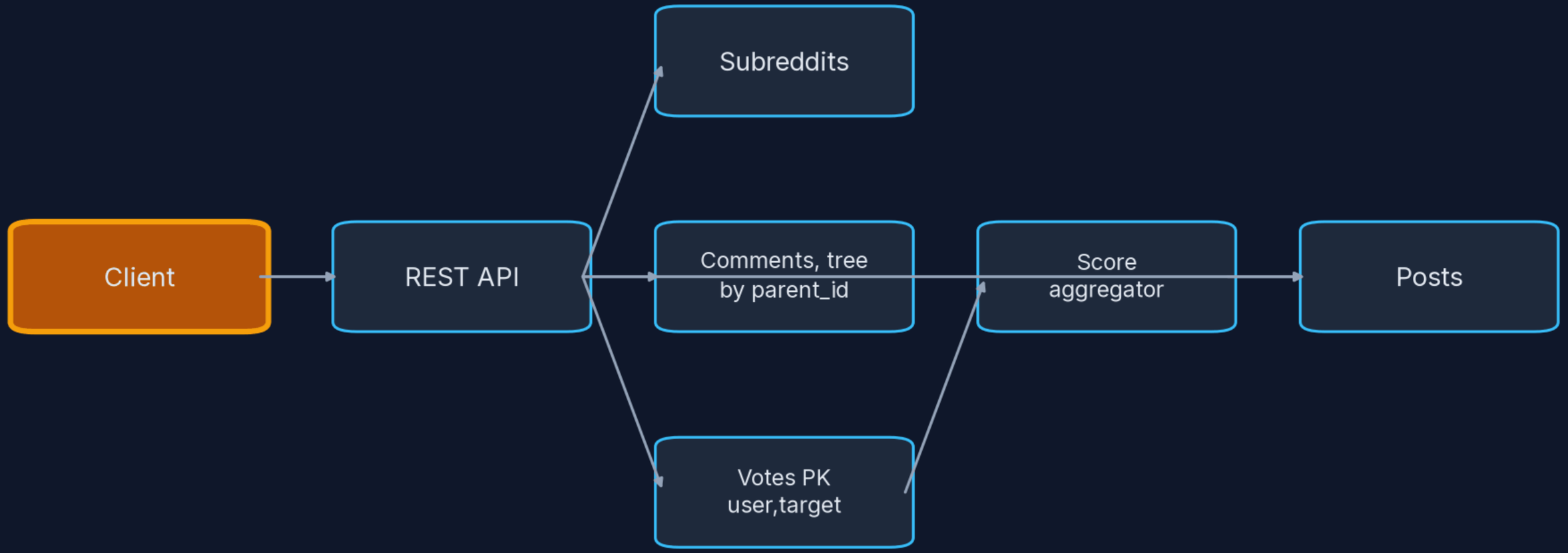
Capacity: worked out

- Read:write ≈ 100:1
- Say 50M DAU × 20 votes = 1B votes/day
- 1B ÷ 86,400 ≈ 12k votes/s
- Batching per post turns hot rows into 1 write/s

High-level architecture

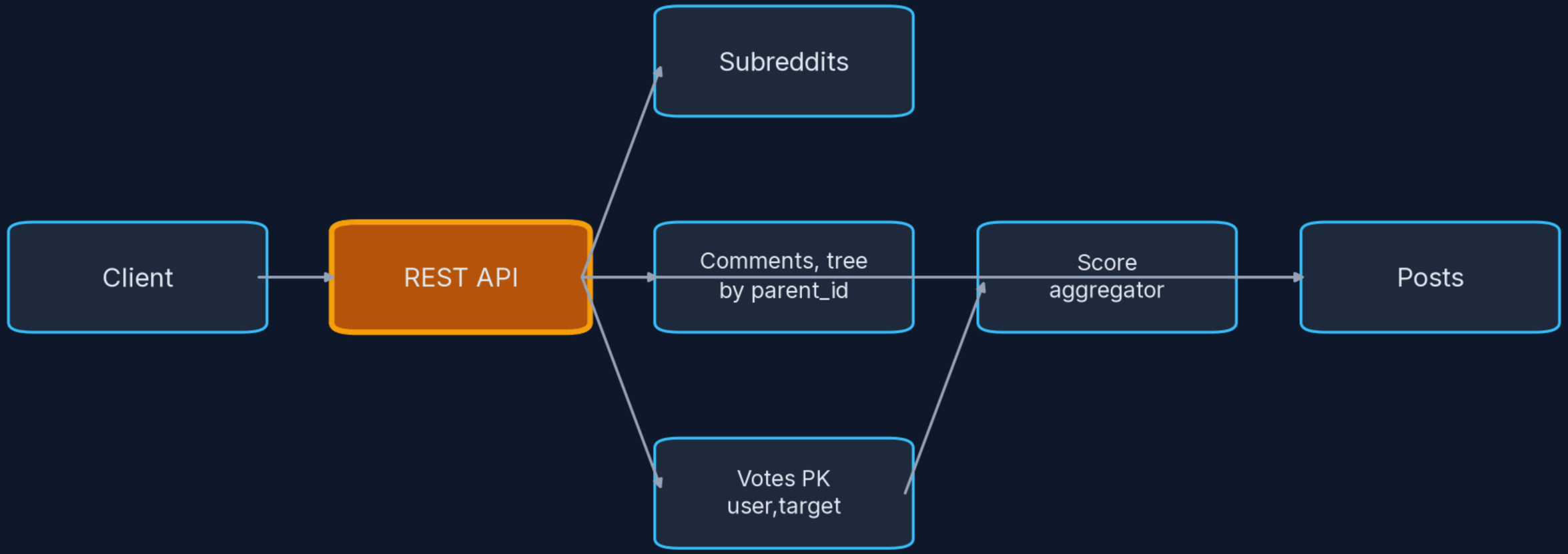


Request flow: Upvote a post



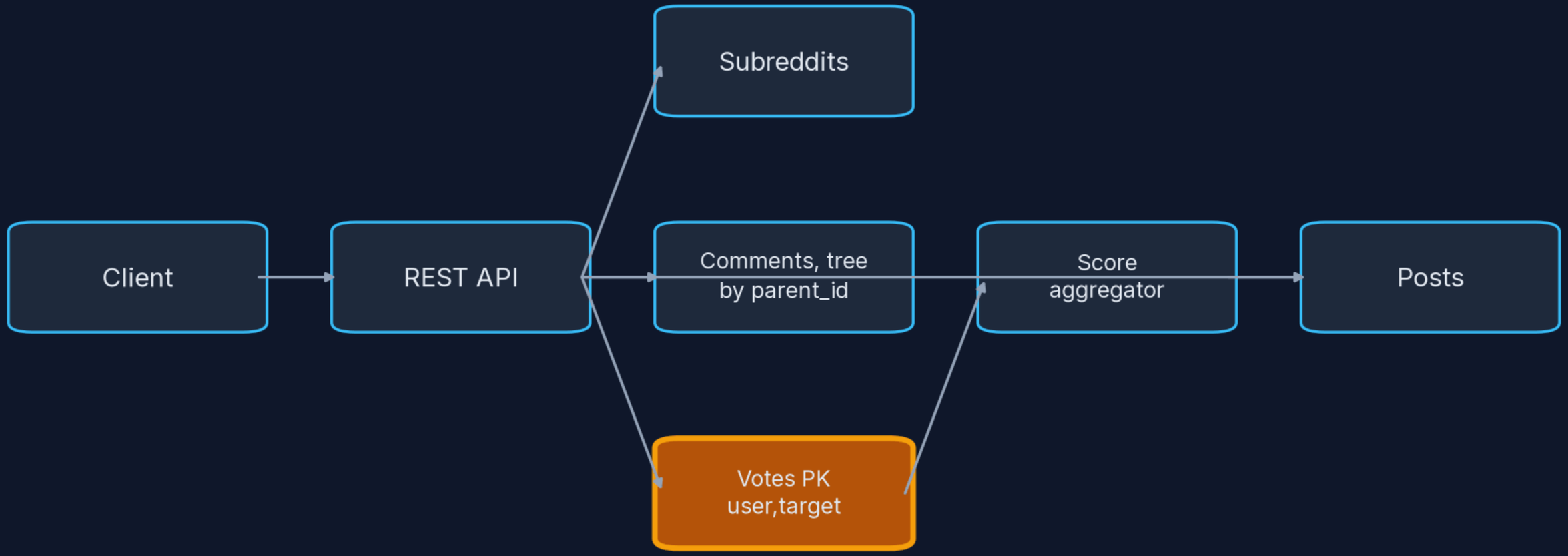
1. User taps upvote

Request flow: Upvote a post



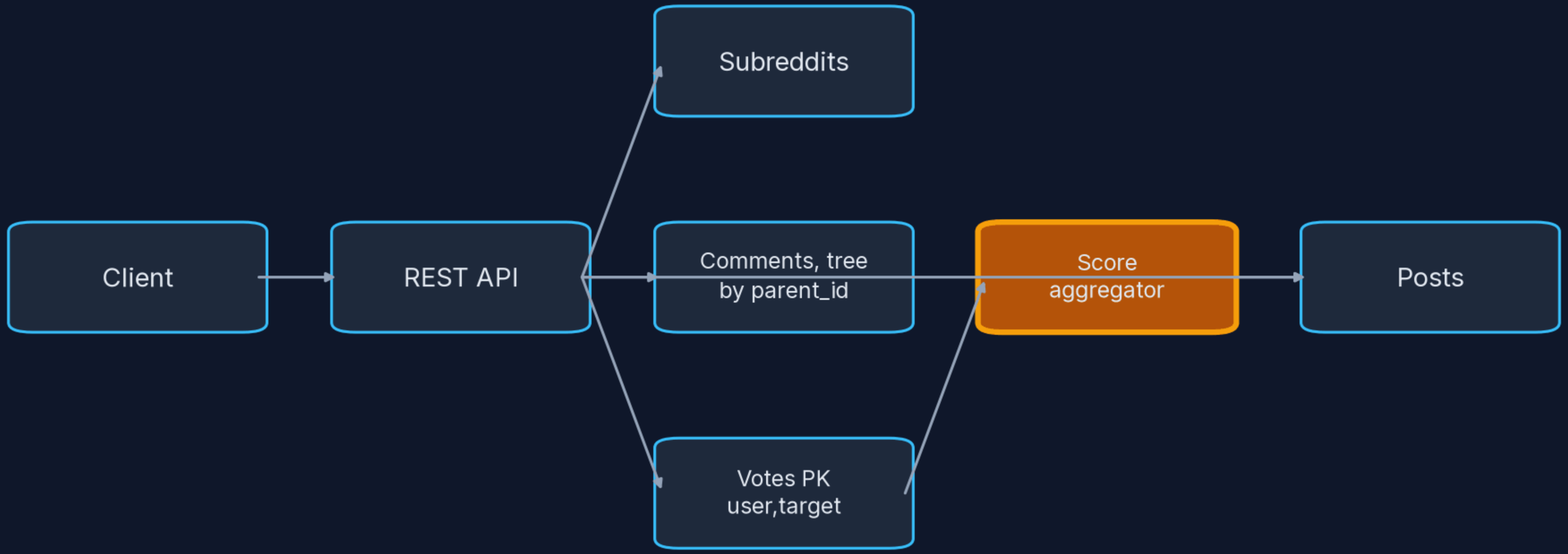
2. API validates

Request flow: Upvote a post



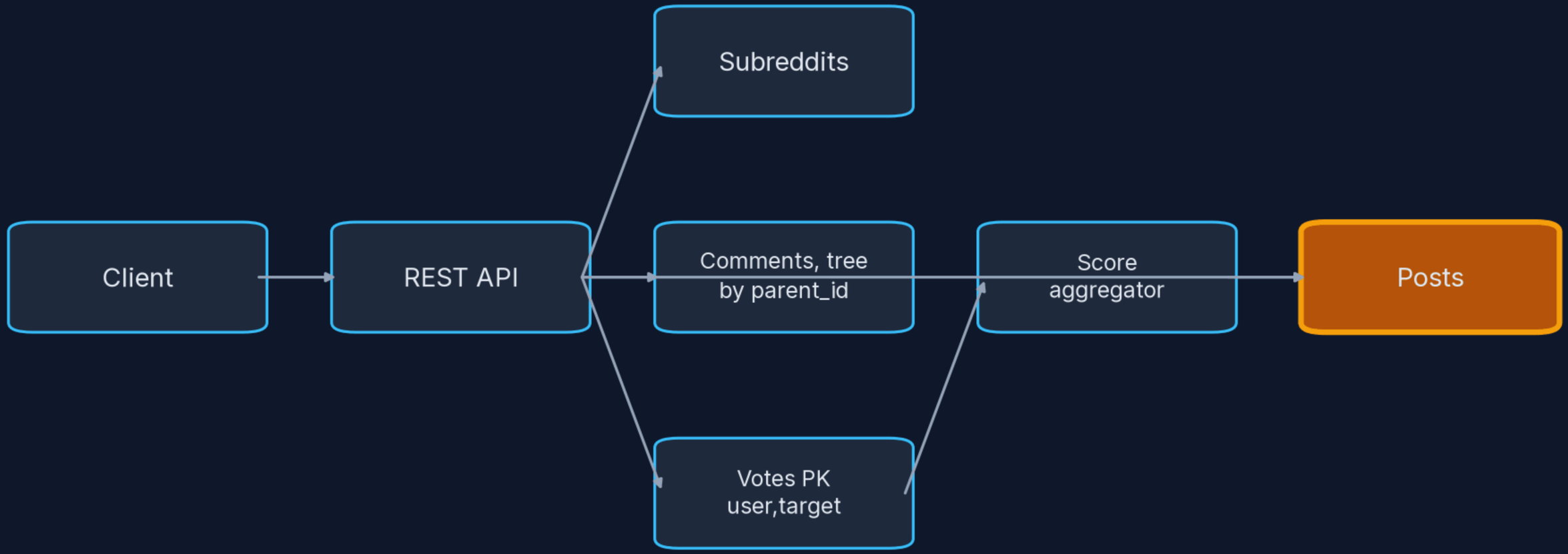
3. Upsert the vote row

Request flow: Upvote a post



4. Aggregate score changes

Request flow: Upvote a post



5. Post score updated

Data model

- Subreddit(id, name, description, created_by)
- Post(id, subreddit_id, author_id, title, body, score, comment_count, created_at, edited_at, deleted_at)
- Comment(id, post_id, parent_id, author_id, body, score, ...)
- Vote(user_id, target_id, target_type, value in {-1,0,1})

API

```
GET    /v1/subreddits/{name}
GET    /v1/subreddits/{name}/posts?sort=hot|new|top&cursor=..&limit=..
POST   /v1/subreddits/{name}/posts              {title, body}
PATCH  /v1/posts/{postId}                       {body}
DELETE /v1/posts/{postId}
GET    /v1/posts/{postId}/comments?sort=..&depth=..&cursor=..
POST   /v1/posts/{postId}/comments              {parentId?, body}
PUT    /v1/posts/{postId}/vote                  {value: -1|0|1}
PUT    /v1/comments/{commentId}/vote            {value}
```

Votes

- PUT with an absolute value makes votes idempotent; the aggregator applies (new - old) deltas to cached scores asynchronously.

Comment trees

- Fetch top-N roots by score then children to a limited depth; 'load more' via cursor on parent_id.

Pagination

- Opaque cursors (score, id) rather than offsets so lists stay stable while new content arrives.

Key decisions

- PUT vote (idempotent) → safe retries, one vote per user
- Cursor pagination → stable under inserts
- Async score aggregation → avoids hot-row contention
- Comments by parent_id → simple tree loading

Trade-offs & alternatives

- Soft deletes keep threads intact ('[deleted]')
- Denormalised counts are eventually consistent
- PUT for votes vs POST upvote/downvote endpoints: PUT is simpler to make idempotent

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Client, REST API, Subreddits, Posts, Comments, tree by parent_id
- Know your trade-offs and failure modes